Phase-Associative Memory: Attention-Free Sequence Modeling via Complex Matrix States

Gowrav Vishwakarma*

March 2026

Abstract

We present Phase-Associative Memory (PAM), a recurrent sequence modeling architecture that replaces softmax attention with complex-valued phase interference over a learned matrix state. Each PAM head maintains a state matrix $S_t \in \mathbb{C}^{d \times d}$, updated via outer products of complex-valued key–value pairs and retrieved via complex dot products between *scaled* queries and conjugated keys (the same $1/\sqrt{d}$ scaling used in standard attention). The conjugate inner product $K_t^* \cdot \tilde{Q}_t$ (with $\tilde{Q}_t := Q_t/\sqrt{d}$) produces constructive or destructive interference that functions as an implicit attention mechanism without softmax normalization. A Gated State Protection (GSP) mechanism allows the model to selectively freeze state dimensions, retaining important information indefinitely.

PAM admits two equivalent computational forms: a dual (quadratic) form for parallel training and a recurrent form for $O(1)$-per-token inference with fixed $O(d^2)$ memory per head, requiring no KV cache. The entire model operates in complex-valued space with phase-preserving primitives end to end.

We evaluate PAM as the core sequence layer in a 100M-parameter language model trained on WikiText-103. Our reported configuration (**medium-pam-v3**) interleaves channel mixing and PAM in each of 16 blocks, applies complex RoPE to PAM queries and keys for relative position, fuses Q/K/V projections for efficiency, and trains with a higher learning rate and longer warmup than an earlier sequential layout. It achieves **validation perplexity 29.95** after 10 epochs. Against a **matched transformer baseline** (∼100M parameters, same data, tokenizer, sequence length, and training budget), which reaches **27.08** validation PPL, PAM is within ∼**10%**—while being a fundamentally different architecture class (complex-valued, attention-free, fixed recurrent state) with $O(1)$-per-token inference and no KV cache. We present PAM as a promising research direction rather than a finished system, and discuss both properties that may matter at scale and limitations that remain. **Code:** `https://github.com/gowrav-vishwakarma/qllm2`.

1 Introduction

The transformer architecture has become the dominant paradigm for language modeling, with its core mechanism—scaled dot-product attention—providing flexible, content-dependent information routing across sequences. However, attention's $O(T^2)$ cost per layer during inference and its linearly growing KV cache have motivated substantial research into alternative sequence processing mechanisms, including state-space models (e.g. S4 [4], Mamba [3]), linear attention variants, and recurrent architectures with selection mechanisms.

This paper contributes to that line of work from an unusual angle: complex-valued computation with phase-preserving operations throughout the model. The core hypothesis is that representing

*Code: `https://github.com/gowrav-vishwakarma/qllm2`

tokens as complex vectors and performing retrieval via phase interference—constructive alignment between query and key phases amplifies recall, destructive misalignment suppresses it—can serve as a natural alternative to softmax-normalized attention.

PAM emerged from a multi-stage research process:

1. **V4** introduced the idea of tokens in complex phase space with wave-style interference, establishing the non-transformer direction.

2. **V5** identified and fixed a critical flaw: V4 passed complex representations through real-valued nonlinearities (GELU, sigmoid) that destroyed phase information. V5 introduced phase-preserving primitives (modReLU, ComplexGatedUnit, ComplexNorm) and showed that mathematical consistency materially improved results.

3. **V6** built on V5's phase-preserving foundation with a multi-timescale complex SSM backbone. Initial experiments with vector-state SSMs showed that associative capacity was limited by state interference: binding multiple facts into a single vector caused catastrophic collisions.

4. **PAM** solves the interference problem by upgrading the state from a vector ($\mathbb{C}^d$) to a matrix ($\mathbb{C}^{d\times d}$), providing $O(d^2)$ associative capacity per head.

5. **Layout and position encoding (medium-pam-v3)**: Interleaving ComplexGatedUnit and PAM in every block (rather than stacking all CGU layers before all PAM layers) improves early sequence mixing. Complex RoPE on PAM Q and K supplies explicit relative-position structure; fused Q/K/V projections and a block-real GEMM implementation preserve the same mathematics with fewer kernel launches. Together with an adjusted AdamW schedule, these changes yield our best reported WikiText-103 result (Section 5).

This paper focuses on PAM and the evidence from validated training runs. We are explicit about what PAM has demonstrated and what remains open.

1.1 Contributions

- **Phase-Associative Memory (PAM)**: a recurrent layer with complex matrix state, data-dependent decay, and retrieval via complex phase interference—no softmax, no KV cache.

- **Gated State Protection (GSP)**: a learned per-dimension freeze gate that allows the model to retain important state indefinitely.

- **Dual-form training**: an $O(T^2)$ parallel computation equivalent to the $O(T)$ recurrence, enabling efficient GPU training.

- **Interleaved blocks + complex RoPE**: sixteen blocks of [CGU $\rightarrow$ PAM] with RoPE on PAM Q, K (and optional implementation fusion) as the **medium-pam-v3** recipe.

- **Empirical validation**: WikiText-103 validation perplexity **29.95** at $\sim$100M parameters (**medium-pam-v3**), with coherent multi-sentence generation. We also report the earlier **sequential** medium-pam run (**38.95**) for comparison (Section 5.1).

- **Matched transformer baseline**: a standard transformer ($\sim$100M parameters) trained with the same data, tokenizer, sequence length, and hyperparameters reaches **27.08** validation PPL, providing a rigorous point of comparison (Section 5.3).

1.2 Scope and honest framing

We do not claim that PAM matches or exceeds transformers. A matched transformer baseline ($\sim$100M parameters, identical data and training budget) reaches **27.08** validation PPL on WikiText-103; our **medium-pam-v3** reaches **29.95**, a gap of $\sim$10%. What we do claim is narrower: PAM is a genuinely different architecture—complex-valued, attention-free, recurrent with $O(1)$ inference cost per token and no KV cache—that has reached a performance level where continued investigation is justified. We present this as a research contribution, not a production system.

2 Background and Related Work

2.1 Transformers and attention

The transformer [12] computes attention as $\text{softmax}(QK^\top/\sqrt{d})V$, where queries, keys, and values are linear projections of the input. This is powerful but costs $O(T^2)$ per layer and requires a KV cache that grows linearly with sequence length during inference. Rotary position embeddings (RoPE; Su et al. 10) encode relative position in the $QK^\top$ product by position-dependent rotations in embedding space; our PAM variant applies an analogous **complex** phase rotation to Q and K only (Section 3.4.3). Numerous efficiency variants exist (sparse attention, linear attention, sliding window), but the core mechanism remains fundamentally quadratic.

2.2 State-space models and recurrent alternatives

Modern SSMs (S4, Mamba, Mamba-2, Griffin) process sequences through learned recurrences with diagonal or structured state transitions. Mamba introduced input-dependent selection (selective SSMs), allowing the recurrence to be content-aware. These models achieve $O(T)$ training via parallel scan and $O(1)$ per-token inference with fixed state. PAM shares the $O(1)$ inference property but differs in two key ways: the state is a complex-valued matrix rather than a real or complex vector, and retrieval uses phase interference rather than a learned output projection.

2.3 Linear attention and matrix-state models

Linear attention [6] reformulates attention by removing softmax, yielding a recurrent form with matrix state $S_t = S_{t-1} + V_t K_t^\top$. Recent work (RetNet, GLA, DeltaNet, Mamba-2) has explored decay-weighted variants of this form. PAM is structurally related to this family but operates entirely in complex space: the state update is $S_t = \gamma_t S_{t-1} + V_t \otimes K_t^*$ (outer product with conjugate), and retrieval computes $Y_t = S_t \tilde{Q}_t$ where the dot product $K_t^* \cdot \tilde{Q}_t$ produces phase interference. The complex conjugate in the key projection is not a superficial notational choice—it means that retrieval quality depends on phase alignment between query and stored key, providing a geometric selectivity mechanism that real-valued matrix states lack.

2.4 Complex-valued neural networks

Complex-valued neural networks have been studied for decades [5], with applications in signal processing, physics-informed modeling, and sequence modeling. Key ingredients include complex linear maps, phase-preserving activations like modReLU [1], and unitary recurrent dynamics. Our work inherits this tradition but applies it specifically to language modeling at scale, which has received less attention in the complex-valued neural network literature [11].

2.5 Associative memory

Holographic Reduced Representations [8] showed that circular convolution can bind key–value pairs into a single vector and circular correlation can retrieve them. In Fourier space, binding becomes element-wise complex multiplication and retrieval becomes multiplication by the conjugate. PAM draws on this insight: `cmul(key, value)` is binding, `cmul(query, conj(stored))` is retrieval. However, PAM learned from a failed experiment (Holographic State Binding, Section 6.1) that vector-state binding causes interference when multiple associations accumulate. The matrix state solves this by providing $O(d^2)$ capacity instead of $O(d)$.

3 Method

3.1 Overview

The **medium-pam-v3** backbone interleaves channel mixing and sequence mixing in each block (preset `interleave_pam=True`). The full model is:

```
Tokens -> ComplexEmbed -> ComplexNorm
  -> [ ComplexGatedUnit + residual
       -> PhaseAssociativeLayer + residual ] x 16 blocks
  -> ComplexLinear -> ComplexNorm -> TiedComplexLMHead
```

An earlier **sequential** layout (`medium-pam`) instead ran `[CGU + residual] x 16` followed by `[PAM + residual] x 16`; Section 5.1 reports both.

All operations in the main signal path are complex-valued and phase-preserving. Control signals (gates, decay parameters) may use real-valued projections over magnitude features, but the primary data path never converts complex representations to real-valued intermediate forms.

3.2 Phase-preserving primitives

These primitives, inherited from V5, form the algebraic foundation:

Complex representation. All tensors have shape `[..., dim, 2]`, where the last dimension holds (real, imaginary) components. This is a concrete implementation of $\mathbb{C}^{\text{dim}}$.

Complex linear map. Given weight matrices $W_r, W_i \in \mathbb{R}^{m\times n}$, the complex linear map computes:

$$y_r = W_r x_r - W_i x_i, \quad y_i = W_i x_r + W_r x_i. \tag{1}$$

This is standard complex matrix multiplication. Conceptually it corresponds to four real matrix multiplies; the **training implementation** packs them into a **single block-real GEMM** (max float32 error $\sim 4 \times 10^{-6}$ vs. the unpacked form in our checks), reducing kernel launches without changing the math.

modReLU. A phase-preserving activation that thresholds magnitude while leaving phase untouched:

$$\text{modReLU}(z) = \text{ReLU}(|z| + b) \cdot \frac{z}{|z|} \tag{2}$$

where b is a learned bias. This replaces GELU/ReLU, which would destroy phase information.

ComplexNorm. RMS normalization applied to magnitudes, with phase preserved:

$$\text{ComplexNorm}(z) = s \cdot \frac{|z|}{\text{RMS}(|z|)} \cdot \frac{z}{|z|} \tag{3}$$

where s is a learned scale parameter.

ComplexGatedUnit (CGU). A SwiGLU-style gating block in complex space. The gate magnitude $\sigma(|W_g z|)$ controls how much signal passes, while the gate phase controls what rotation is applied. An up-projection through modReLU provides the nonlinearity:

$$\text{CGU}(z) = W_{\text{down}}\big(\text{gate}_{\text{phase}} \odot \text{modReLU}(W_{\text{up}} z) \cdot \sigma(|W_g z|)\big) \tag{4}$$

3.3 Blocks (interleaved CGU and PAM)

Each of 16 blocks applies **channel mixing** then **sequence mixing** with residual connections and learned scaling:

$$\tilde{z}^{(l)} = z^{(l-1)} + \alpha^{(l)}_{\text{CGU}} \cdot \text{CGU}_l(\text{ComplexNorm}(z^{(l-1)})), \tag{5}$$

$$z^{(l)} = \tilde{z}^{(l)} + \alpha^{(l)}_{\text{PAM}} \cdot \text{PAM}_l(\text{ComplexNorm}(\tilde{z}^{(l)})) \tag{6}$$

where $\alpha^{(l)}_{\text{CGU}}$ is initialized to 1.0 and $\alpha^{(l)}_{\text{PAM}}$ to 0.1. Each CGU uses expansion factor 3 (hidden dimension $3 \times 384 = 1152$). Dropout is applied during training.

3.4 Phase-Associative Memory (PAM)

PAM is the core contribution. It replaces both the recurrent backbone (SSM) and the attention mechanism with a single module.

3.4.1 State representation

Each PAM head h maintains a complex matrix state $S_t^{(h)} \in \mathbb{C}^{d \times d}$, where d is the head dimension. With H heads and head dimension d, the total state capacity is $H \times d^2$ complex values. In our configuration ($H = 6$, $d = 64$), this is $6 \times 64^2 = 24{,}576$ complex values per layer.

3.4.2 Projections and fused QKV

The input $x_t \in \mathbb{C}^D$ (where $D = 384$) is projected into queries, keys, and values. In **medium-pam-v3**, a **single** complex linear map produces a vector of length $3Hd$, which is reshaped into Q_t, K_t, V_t; this is equivalent to three separate maps and does not change the parameter count.

$$[Q_t; K_t; V_t] = W_{\text{QKV}} x_t \quad \Rightarrow \quad Q_t, K_t, V_t \in \mathbb{C}^{H \times d}. \tag{7}$$

3.4.3 Complex RoPE on Q and K

We apply **complex RoPE** to Q and K only (not V, not the residual stream). For each head dimension and position m, Q_m and K_m are multiplied element-wise by precomputed **unit-magnitude** complex factors $e^{im\theta}$ (frequencies θ fixed by dimension, as in standard RoPE). Thus $|Q|, |K|$ are unchanged and phase encodes absolute position; in products $K_i^* \cdot \tilde{Q}_t$, dependence on $(m-n)$ yields **relative** position structure. RoPE is applied **before** forming training-time $QK^{*\top}$ and before each recurrent inference step (with an explicit step offset during generation).

3.4.4 Scaled query for retrieval

Following the attention scaling convention, we define the **scaled query**

$$\tilde{Q}_t := \frac{1}{\sqrt{d}} Q_t. \tag{8}$$

All retrieval dot products in Sections 3.4.8 to 3.4.10 use $\tilde{Q}_t$ (implemented by scaling Q before the complex bilinear form). Unscaled Q_t still appears in projections and RoPE; only the score and matrix–vector product use $\tilde{Q}_t$.

3.4.5 Data-dependent decay

The decay rate γ_t controls how quickly the state forgets. It is computed from the input:

$$dt_t = \text{softplus}(W_{dt} \cdot \text{concat}(x_{t,r}, x_{t,i}) + b_{dt}) \tag{9}$$

where $W_{dt} \in \mathbb{R}^{H \times 2D}$ projects the concatenated real and imaginary parts to one scalar per head, and b_{dt} is initialized to -4.0 (yielding slow initial decay).

3.4.6 Gated State Protection (GSP)

GSP adds a learned protect gate p_t that can freeze state dimensions:

$$p_t = \sigma(W_p \cdot |x_t| + b_p), \quad b_p \text{ initialized to } -3.0 \tag{10}$$

$$\gamma_t = e^{-dt_t} \cdot (1 - p_t) + p_t. \tag{11}$$

When $p_t \to 1$, $\gamma_t \to 1$ and the state is frozen regardless of the decay signal. When $p_t \to 0$, the decay proceeds normally. The protect gate bias is initialized to -3.0 ($\sigma(-3) \approx 0.047$), so protection starts near zero and must be learned.

The value signal is also modulated by the protect gate:

$$V'_t = V_t \cdot (1 - p_t). \tag{12}$$

This prevents new values from overwriting protected state dimensions.

3.4.7 State update

The state evolves as:

$$S_t = \gamma_t \cdot S_{t-1} + V'_t \otimes K^*_t \tag{13}$$

where $\otimes$ denotes complex outer product and K^*_t is the complex conjugate of the key. This is a decay-weighted linear recurrence over matrices.

3.4.8 Retrieval via phase interference

Retrieval computes:

$$Y_t = S_t \, \tilde{Q}_t. \tag{14}$$

Expanding the state, with $\tilde{Q}_t = Q_t/\sqrt{d}$,

$$Y_t = \sum_{i \le t} \left(\prod_{j=i+1}^{t} \gamma_j \right) \left(K^*_i \cdot \tilde{Q}_t \right) V'_i. \tag{15}$$

The term $K_i^* \cdot \tilde{Q}_t$ is a complex dot product. Its magnitude depends on phase alignment between the stored key K_i and the current query Q_t:

- **Constructive interference**: when K_i and $\tilde{Q}_t$ have aligned phases, $|K_i^* \cdot \tilde{Q}_t|$ is large, amplifying retrieval of the corresponding V_i'.
- **Destructive interference**: when phases are misaligned, the term is small or cancels, suppressing retrieval.

This is the mechanism that replaces softmax attention. It is not learned separately—it is an intrinsic property of complex dot products.

3.4.9 Dual form (training)

During training, we avoid materializing the $d \times d$ matrix sequentially. Instead, we compute the output in $O(T^2)$ time using dense matrix multiplications:

1. Compute the decay matrix $D \in \mathbb{R}^{T \times T}$ in log space:

$$\log D[t, i] = \sum_{j=i+1}^{t} \log \gamma_j \tag{16}$$

   using cumulative sums for numerical stability.

2. Apply a causal mask: $D[t, i] = 0$ for $i > t$.
3. Apply RoPE to Q and K at each position (Section 3.4.3), then compute the complex attention-like matrix:

$$W = \tilde{Q}\, K^{*\top} = (Q/\sqrt{d})\, K^{*\top}. \tag{17}$$

4. Apply decay: $A = W \odot D$.
5. Compute output: $Y = A \cdot V'$.

This is mathematically equivalent to the recurrent form but parallelizes across the sequence dimension. For $T = 2048$, this is efficient on modern GPUs.

At the end of training forward passes, we also compute the final recurrent state S_T from the decay matrix, so that generation can continue from a prompt:

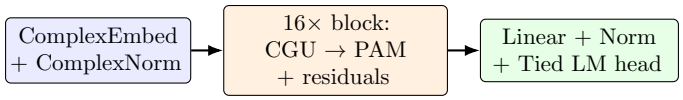
$$S_T = \sum_{i=1}^{T} D[T, i] \cdot (V_i' \otimes K_i^*). \tag{18}$$

3.4.10 Recurrent form (inference)

During autoregressive generation, each token is processed in $O(1)$ amortized time per layer (constant in sequence length; work per token is $O(Hd^2)$):

1. Compute Q_t, K_t, V_t', γ_t from the input; apply RoPE to Q_t, K_t at the current absolute position.
2. Update state: $S_t = \gamma_t \cdot S_{t-1} + V_t' \otimes K_t^*$.
3. Retrieve: $Y_t = S_t\, \tilde{Q}_t = S_t\, (Q_t/\sqrt{d})$.

The state $S \in \mathbb{C}^{H \times d \times d}$ is fixed-size and does not grow with sequence length.



(a) **medium-pam-v3** backbone: interleaved channel mixing (CGU) and sequence mixing (PAM) per block.

(b) Single PAM head (inference path): fused QKV, complex RoPE on Q, K, decay and GSP, outer-product write to matrix state, retrieval with scaled query $\tilde{Q}_t = Q_t/\sqrt{d}$, output projection.

Figure 1: Architecture diagrams for Phase-Associative Memory (**medium-pam-v3**).

3.4.11 Stacking within a block

Within each block, PAM follows CGU with pre-normalization, residual, and learned layer scale (Section 3.3). A final ComplexNorm is applied before the output head.

3.5 Output head

The output projection computes logits via a tied complex inner product with the embedding table:

$$\text{logits} = \text{Re}(z_{\text{out}} \cdot \text{conj}(E)) \tag{19}$$

where E is the complex embedding matrix (shared with the input embedding). Concretely:

$$\text{logits} = z_{\text{out},r} \cdot E_r^\top + z_{\text{out},i} \cdot E_i^\top. \tag{20}$$

This weight tying reduces parameters and maintains algebraic consistency.

4 Experimental Setup

4.1 Dataset

We train and evaluate on WikiText-103 [7], a standard language modeling benchmark containing approximately 103 million tokens of Wikipedia text. We use the official train/validation splits, tokenized with the GPT-2 BPE tokenizer (vocabulary size 50,257).

4.2 Model configuration

Primary reported model (medium-pam-v3). See Table 1.

4.3 Training details

Hyperparameters below match **medium-pam-v3** (preset `medium-pam-v3` in `v6/config.py`). The earlier **sequential** `medium-pam` run used learning rate 3×10^{-5} and 500 warmup steps (Section 5.1).

4.4 Generation settings (qualitative evaluation)

During training, generation samples are logged every 5,000 steps using: temperature 1.0; top-k 50; top-p 0.9; repetition penalty 1.2.

Table 1: medium-pam-v3 configuration (reported).

Parameter	Value
Complex dimension (D)	384
Blocks	16 (each: CGU $\rightarrow$ PAM)
Interleave CGU + PAM	yes (`interleave_pam=True`)
CGU expansion factor	3
PAM heads (H)	6
PAM head dimension (d)	64
GSP	enabled
PAM RoPE on Q, K	yes (`pam_rope=True`)
PAM QK phase norm	**off** (`pam_qk_norm=False`; see Section 5.1)
Fused QKV projection	yes (`pam_fused_qkv=True`)
Sequence length	2048
Total parameters	$\sim$100.4M
Working/internal memory	disabled
Softmax attention	disabled

5 Results

5.1 Main result

medium-pam-v3 (interleaved + RoPE + fused QKV, no QK phase norm) reaches validation perplexity **29.95** after 10 epochs on WikiText-103 (single run, RTX 4090). Figures 2 and 3 and Table 3 summarize training progress.

Total wall time **50,714 s ($\sim$14.1 h)**. Throughput **$\sim$23k tokens/second** average.

QK phase normalization ablation. We briefly trained with per-element unit normalization of Q and K before the dot product. Validation loss decreased, but **generation** collapsed into severe lexical repetition by mid-training; we **stopped** the run during **epoch 5**. PAM scores are not softmax-normalized; removing Q/K magnitude removes a degree of freedom that appears important for non-softmax retrieval. We **do not** use QK phase norm in the reported preset (see project log `EXPERIMENTS_V6_PART2.md`, Bug 8).

Earlier configuration: sequential medium-pam. An earlier **sequential** stack (16 CGU layers then 16 PAM layers), no RoPE, learning rate 3×10^{-5}, 500 warmup steps, achieved **38.95** validation PPL after 10 epochs. Epochs 1–3 had correct PPL but a train/inference path bug affecting **generation** only; the run was resumed from the epoch 3 checkpoint with the fix. PPL for epochs 4–10 is shown in Table 4.

Wall time for that continuation was **$\sim$9.9 h**; throughput **$\sim$23,500 tok/s**.

The **29.95** vs **38.95** comparison is **not** a controlled ablation: layout, RoPE, fused QKV, block GEMM packaging, and **training hyperparameters** changed together.

Hybrid PAM + sparse attention (PIA). We tested **medium-pam-v3-pia**, which adds Phase Interference Attention (PIA)—a windowed (w=256), sparse attention layer inserted every 4th block—on top of the medium-pam-v3 architecture. This hybrid achieved **30.01** validation PPL, marginally **worse** than pure PAM (29.95), while increasing wall time to 16.4 h ($\sim$20k tok/s). The

Table 2: Training hyperparameters (medium-pam-v3).

Parameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	0.01
Warmup steps	1000
LR schedule	warmup + cosine decay
Batch size	3
Epochs	10
Gradient clipping	1.0
Precision	automatic mixed precision (bf16)
Hardware	single NVIDIA RTX 4090 (24GB)
Compilation	torch.compile (default mode)
Initialization	orthogonal (complex linear maps)

Table 3: medium-pam-v3: training and validation perplexity by epoch (WikiText-103).

Epoch	Train PPL	Val PPL
1	123.86	57.94
2	53.87	43.83
3	44.88	38.69
4	40.39	35.88
5	37.42	33.82
6	35.13	32.25
7	33.26	31.22
8	31.78	30.40
9	30.66	30.01
10	30.02	**29.95**

result suggests that PAM's recurrent sequence mixing already captures the information that sparse attention would provide at this scale, and the added complexity does not help.

5.2 Ablation across V6 architectures

Table 5 shows the progression of V6 architectures on WikiText-103 (10 epochs, sequence length 2048 where noted). Rows are **not** a single-variable grid.

Key observations:

- **GSP helps**: Adding Gated State Protection to the SSM baseline reduces perplexity from 44.47 to 41.67 (6.3% improvement).
- **HSB hurts**: Holographic State Binding causes a regression to 43.54. This supports the interference hypothesis: vector states lack capacity for multiple simultaneous associations.
- **PAM works**: Matrix-state PAM improves over the GSP SSM baseline; **medium-pam-v3** further improves over sequential **medium-pam** (layout + RoPE + recipe), reaching **29.95**.

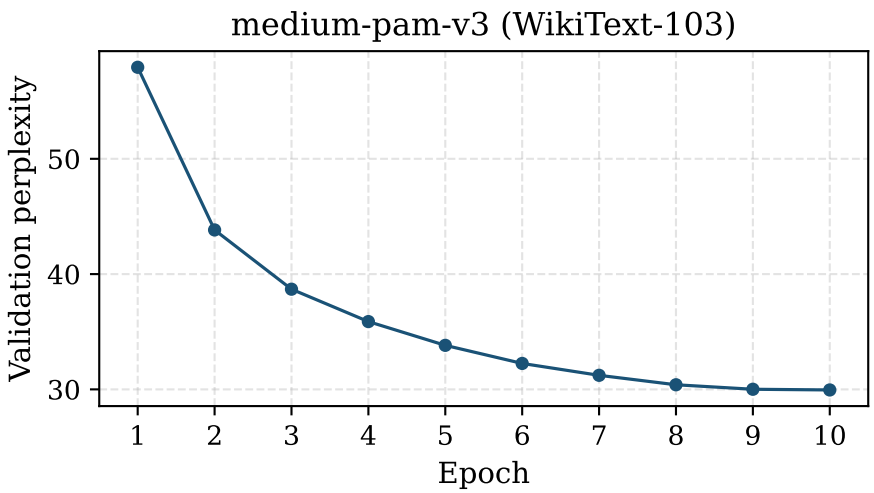


Figure 2: Validation perplexity vs. epoch for **medium-pam-v3** (same data as Table 3).

Table 4: Sequential medium-pam: epochs 4–10 after resume (WikiText-103).

Epoch	Train PPL	Val PPL
4	54.03	47.19
5	48.55	43.55
6	45.26	41.43
7	43.14	40.11
8	41.76	39.34
9	40.91	39.02
10	40.50	**38.95**

- **PIA adds no benefit**: Adding sparse Phase Interference Attention (windowed, every 4th block) to medium-pam-v3 yields 30.01, marginally **worse** than pure PAM (29.95). PAM's sequence mixing appears sufficient without supplemental attention.
- **Transformer gap**: A matched transformer baseline reaches **27.08**, a ∼10% advantage over medium-pam-v3. See Section 5.3 for detailed analysis.

5.3 Comparison to matched transformer baseline

To provide a rigorous comparison, we trained a standard transformer with ∼100.3M parameters on the same WikiText-103 data, tokenizer (GPT-2 BPE), sequence length (2048), batch size (3), learning rate (1×10^{-4}), warmup (1000 steps), and hardware (single RTX 4090). The transformer uses $d_{\text{model}} = 672$, 12 layers, 12 attention heads, and $d_{\text{ff}} = 2688$.

medium-pam-v3 reaches **29.95** validation PPL vs. the matched transformer's **27.08**—a gap of ∼10%. The transformer also trains ∼4× faster in wall time (3.5 h vs. 14.1 h) and throughput (∼96k vs. ∼23k tok/s). The throughput gap is expected: complex linear maps require 4× the multiply-adds of real linear maps, and PAM uses pure PyTorch with no custom kernels.

For external reference, the commonly cited GPT-2 [9] validation PPL on WikiText-103 is ∼31 at 124M parameters (test PPL ∼14.84 uses a different sliding-window protocol and is not directly

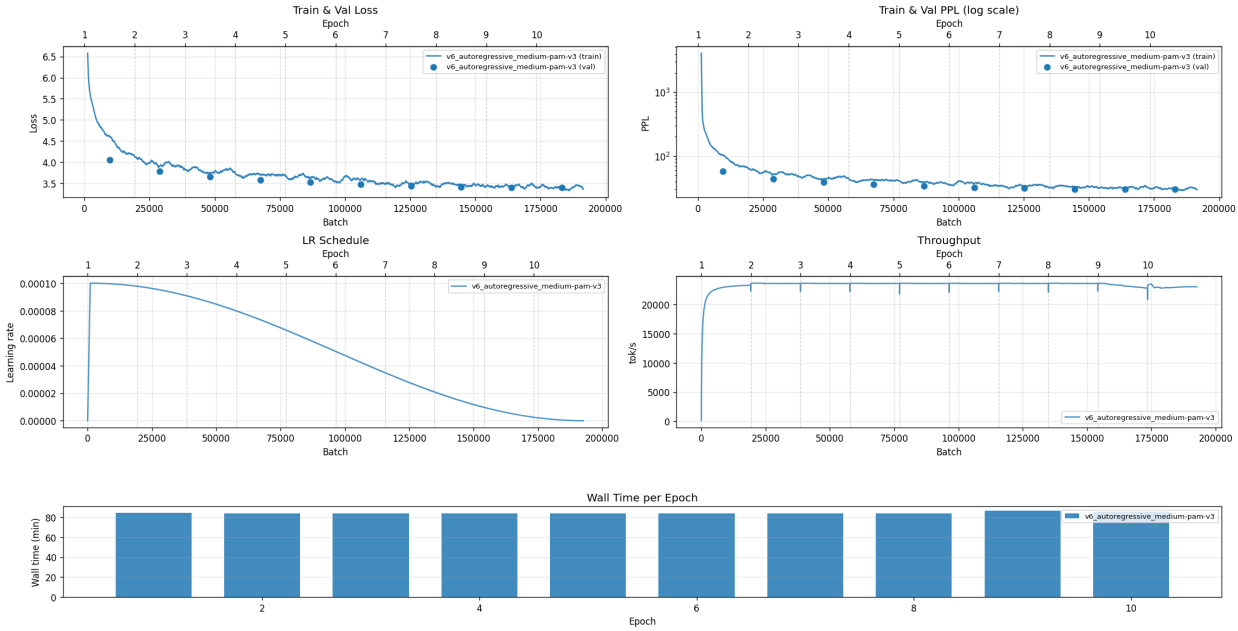


Figure 3: Live training monitor for the reported run (git `31397f0`; wall 50714.3 s; best val PPL **29.95**). Logs under `logs/v6//medium_pam_v3_rope_wikitext103_20260319_231524_31397f0/`.

comparable).

The transformer converges faster and maintains a consistent advantage throughout training (Table 7). Both models use the same epoch count and see the same total tokens.

Structural differences remain important despite the PPL gap:

- **Inference cost**: The transformer requires $O(T)$ work per token (KV cache); PAM is $O(1)$ per token with fixed-size state (per-layer work $O(Hd^2)$, independent of sequence length).
- **Memory footprint**: The transformer's KV cache grows linearly with sequence length; PAM's state is fixed at 49,152 floats per layer regardless of T.
- **Architecture maturity**: Transformers benefit from years of optimization (FlashAttention, fused kernels, etc.); PAM is a first-generation PyTorch implementation with no custom CUDA kernels.

5.4 Generation quality

medium-pam-v3 (epoch 10) produces coherent multi-sentence text; factual accuracy is not guaranteed.

Prompt: `In 1923 , the University of`

Generated text (medium-pam-v3, epoch 10):

> In 1923 , the University of Illinois at Urbana @-@ Urdu said it was " an easy choice to do something in its own right . " The university also claimed the first students from Wisconsin had to be replaced by a more " good student " due to a lack of funds .

Table 5: V6 architecture progression on WikiText-103 (validation perplexity). The transformer baseline uses the same data, tokenizer, sequence length, and training budget.

Model	Architecture	Params	Val PPL
medium-rebalanced	CGU + vector-state SSM	58.4M	44.47
medium-rebalanced-gsp	CGU + SSM + GSP	63.2M	41.67
medium-rebalanced-hsb	CGU + SSM + GSP + HSB	75.0M	43.54
medium-pam	CGU ×16 then PAM ×16 + GSP	100.4M	38.95
medium-pam-v3-pia	medium-pam-v3 + sparse PIA (every 4 blocks)	∼100.4M	30.01
medium-pam-v3	**16×[CGU→PAM] + GSP + RoPE + fused QKV**	**∼100.4M**	**29.95**
Transformer (ours)	Standard transformer (12 layers, 12 heads)	∼100.3M	27.08

Table 6: Matched comparison: transformer baseline vs. PAM (same data, tokenizer, seq_len, LR, hardware).

Model	Type	Params	Val PPL	Wall time	Throughput
GPT-2 (cited)	Transformer	124M	∼31	–	–
Transformer (ours)	Transformer	∼100.3M	**27.08**	3.5 h	∼96k tok/s
PAM (ours)	**Complex recurrent**	**∼100.4M**	**29.95**	**14.1 h**	**∼23k tok/s**

Quality metrics at final epoch: 3-gram repetition rate **0.034**, 4-gram repetition rate **0.011**, zero restarts, unique token ratio **0.703**. The sample shows section-like structure and WikiText-style tokenization artifacts; it does **not** exhibit the list-repetition failure mode of the QK-norm ablation.

Sequential medium-pam (epoch 10) for the same prompt (different run, see Section 5.1):

> In 1923 , the University of Missouri and the University of Michigan was also established in 1926 . In 1928 , the University of Michigan opened its current campus with the school 's first campus opening at St. Louis Road on the northern end of Lake Michigan in 1929 .

Quality metrics (sequential run): 3-gram rep. 0.051, 4-gram rep. 0.020, unique token ratio 0.624.

The text is grammatically coherent and shows structural awareness (dates, proper nouns, institutional language). Factual accuracy is not reliable—this is expected at this scale.

5.5 Earlier validation: TinyStories

Before WikiText-103, the broader V6 architecture family was validated on TinyStories [2]:

Table 7: Epoch-by-epoch validation PPL: matched transformer vs. medium-pam-v3.

	Transformer (ours)		PAM (ours)	
Epoch	Train PPL	Val PPL	Train PPL	Val PPL
1	137.77	53.74	123.86	57.94
2	52.81	39.42	53.87	43.83
3	42.74	34.76	44.88	38.69
4	38.27	31.96	40.39	35.88
5	35.54	30.39	37.42	33.82
6	33.41	29.02	35.13	32.25
7	31.65	28.09	33.26	31.22
8	30.28	27.46	31.78	30.40
9	29.29	27.15	30.66	30.01
10	28.75	**27.08**	30.02	**29.95**

Model	Config	Params	Dataset	Val PPL
V6 small-matched	CGU + SSM, no memory	28.7M	TinyStories (full)	5.50
V6 small-matched	CGU + SSM, WM=16	29.2M	TinyStories (full)	2.23

These results established that the phase-preserving, attention-free architecture family can learn effectively. The TinyStories experiments also revealed that working memory capacity is a critical control variable: too much memory induces memorization, while zero memory produces cleaner but higher-perplexity generations.

6 Analysis

6.1 Why HSB failed and PAM works

The progression from SSM to HSB to PAM illustrates a specific failure mode and its solution.

Holographic State Binding (HSB) attempted to inject key–value associations into the SSM's vector state using complex multiplication (the HRR binding operation). Mathematically, multiple bindings are superposed additively in the same d-dimensional vector. When only a few associations are present, retrieval via conjugate multiplication works. When many associations accumulate over a long sequence, they interfere destructively—the signal-to-noise ratio is often discussed heuristically as degrading with the number of superposed associations in HRR-style models (e.g. as $O(1/\sqrt{n})$ scaling in classical analyses of superposition).

PAM solves this by replacing the vector state with a matrix state. The outer product $V_t \otimes K_t^*$ writes each association into a separate subspace of the $d \times d$ matrix. Multiple associations can coexist without interfering because they occupy different rank-1 components of the matrix. The capacity is $O(d^2)$ per head rather than $O(d)$.

This diagnosis—vector state causes interference, matrix state fixes it—is supported by the experimental evidence: HSB regresses from the SSM baseline, while PAM improves over it.

6.2 Phase interference as implicit attention

In standard attention, the softmax over $QK^\top/\sqrt{d}$ produces a probability distribution that weights values. In PAM, the complex dot product $K_i^* \cdot \tilde{Q}_t$ serves an analogous role:

$$\text{Attention: } \alpha_{ti} = \frac{\exp(Q_t \cdot K_i/\sqrt{d})}{\sum_j \exp(Q_t \cdot K_j/\sqrt{d})} \tag{21}$$

$$\text{PAM: } w_{ti} = \left(\prod_{j=i+1}^{t} \gamma_j\right) \cdot (K_i^* \cdot \tilde{Q}_t). \tag{22}$$

The key differences:

1. PAM weights are **complex-valued**, not positive real. This means retrieval can involve both addition and cancellation, potentially providing richer composition.
2. PAM weights are **not normalized**. There is no softmax or equivalent. This removes a nonlinearity but also removes the guarantee that weights sum to 1.
3. PAM weights include a **decay factor** that naturally downweights distant positions, providing a soft locality bias that attention must learn.
4. PAM weights depend on **phase alignment**, a geometric property of complex vectors that standard real-valued attention lacks.

Whether these differences are advantageous at scale is an open question. The current evidence is that they produce a trainable system with reasonable perplexity, but we cannot yet determine whether phase interference provides benefits beyond what real-valued matrix states achieve.

6.3 Computational cost

Training: The dual form computes $W = \tilde{Q}K^{*\top}$ (after RoPE on Q, K) as a $T \times T$ matrix per head, giving $O(T^2Hd)$ cost per layer—the same order as standard attention. For $T = 2048$, this is efficient on modern GPUs. **medium-pam-v3** training throughput is approximately **23k tokens/second** on a single RTX 4090 (wall time ∼**14.1 h** for 10 epochs). By comparison, our matched transformer baseline achieves ∼**96k tokens/second** (wall time ∼3.5 h), a ∼4× throughput advantage. This gap is expected: complex linear maps require 4× the multiply-adds of real linear maps, and the PAM implementation uses no custom CUDA kernels.

Inference: The recurrent form processes each token with $O(Hd^2)$ work per layer (one matrix–vector multiply per head). With $H = 6$ and $d = 64$, this is 24,576 complex multiply-adds per layer. The state is $H \times d \times d \times 2$ floats $= 6 \times 64 \times 64 \times 2 = 49{,}152$ floats per layer, regardless of sequence length.

For comparison, a transformer's KV cache stores $2 \times H \times d \times T$ values per layer, growing linearly with T. At $T = 2048$ with our baseline's configuration (H=12, d=56), the cache per layer is $2 \times 12 \times 56 \times 2048 = 2{,}752{,}512$ values. PAM's state per layer is fixed at 49,152 values, a factor of ∼56× smaller at this sequence length, and the advantage grows with longer sequences.

6.4 The role of GSP

GSP's protect gate allows the model to selectively override decay, retaining specific state dimensions indefinitely. This is conceptually similar to the forget gate in LSTMs but operates at the level of

the entire $d \times d$ state matrix per head. Empirically, GSP improves the SSM baseline from 44.47 to 41.67 perplexity, and PAM uses GSP by default.

The protect gate also modulates the value signal ($V' = V \cdot (1 - p)$), preventing new values from overwriting protected dimensions. This creates a natural partitioning of state into "active" dimensions (low protection, normal read/write) and "frozen" dimensions (high protection, retained over long spans).

7 Limitations

All results in this paper (PAM and the matched transformer baseline) come from **single** training runs. We have not measured variance across seeds or conducted full architecture search. The ∼10% gap between PAM (29.95) and the matched transformer (27.08) should be interpreted with this caveat.

100M parameters and WikiText-103 are modest by current standards. We do not know how PAM scales to billions of parameters or web-scale corpora. The $O(d^2)$ state per head could become a memory concern at very large head dimensions, though for the current $d = 64$ it is negligible.

We report perplexity and qualitative generation only. Standard downstream benchmarks (LAMBADA, HellaSwag, etc.) have not been evaluated.

PAM is implemented in pure PyTorch with no custom CUDA kernels. The dual form uses standard matrix multiplications, which are efficient, but the recurrent form's sequential loop is not optimized. Custom kernels for the state update and retrieval could improve inference throughput substantially.

Complex linear maps require 4× the multiply-adds of real linear maps. The reference implementation is equivalent to four real matrix multiplies; **training** uses a **block-real GEMM** that is numerically equivalent (max float32 diff $\sim 4 \times 10^{-6}$ in our checks). Whether the representational advantages of complex computation compensate for this overhead at scale is unresolved.

The V6 codebase includes working memory, internal memory, persistent memory, expert memory, and session memory modules. The **medium-pam-v3** run reported here disables all of these (no working memory, no internal memory) to isolate PAM's contribution. Whether combining PAM with memory modules improves results is an open question.

8 Future Work

At 100M parameters, our matched transformer baseline outperforms PAM by ∼10% on validation PPL. The most immediate scaling question is whether this gap widens, narrows, or holds at larger scales. We plan to train larger PAM models (300M–1B parameters) alongside matched transformer baselines on the same data and compute budget.

The recurrent inference form's sequential loop is a clear optimization target. A fused CUDA kernel for the complex matrix state update and retrieval could bring PAM's wall-clock inference speed closer to its theoretical advantage.

PAM's fixed-size state and absence of KV cache make it theoretically well-suited for long sequences. We plan to evaluate on PG-19 and other long-document benchmarks to test whether the $O(d^2)$ state capacity is sufficient for practical long-context modeling.

The V6 codebase includes a memory hierarchy (working, internal, persistent, expert) with phase-coherence retrieval. These modules were disabled for the PAM evaluation to isolate the core contribution. Integrating PAM with working memory and evaluating on tasks that require explicit fact storage is a natural next step.

PAM could be combined with sparse attention layers to create a hybrid architecture. Our initial experiment with PIA (Phase Interference Attention, windowed, every 4th block) showed no improvement over pure PAM at this scale (Section 5.2). However, alternative hybrid strategies—such as full (non-windowed) attention at select layers, or standard softmax attention rather than phase-based—remain unexplored.

Prior hybrid (V5 lineage). In this research line, the earlier **V5** codebase first combined end-to-end phase-preserving complex sequence modeling with **sparse** PhaseAttention (sliding window, every K layers) and trained it to strong TinyStories perplexity, establishing a working complex-phase–limited-attention stack before the attention-free PAM direction (see `v5/EXPERIMENTS.md`).

The shared backbone design in V6 was built to support both autoregressive and diffusion modes on the same architecture. Extending PAM to image generation or multimodal modeling is architecturally straightforward but experimentally unvalidated.

9 Conclusion

Phase-Associative Memory is a recurrent sequence modeling architecture that processes information through complex-valued phase interference rather than softmax attention. Its key properties are:

1. A complex matrix state $S \in \mathbb{C}^{H \times d \times d}$ that provides $O(d^2)$ associative capacity per head.
2. Retrieval via complex dot product $K^* \cdot \tilde{Q}$ with $\tilde{Q} = Q/\sqrt{d}$, where phase alignment between query and stored key determines recall strength—constructive interference amplifies, destructive interference suppresses.
3. Gated State Protection that allows selective state freezing.
4. A dual computational form: $O(T^2)$ parallel training, $O(1)$ sequence-length scaling for recurrent state with no KV cache (per-token work $O(Hd^2)$ per layer).

At ∼100M parameters on WikiText-103, **medium-pam-v3** achieves **validation perplexity 29.95**, producing coherent multi-sentence text. A matched transformer baseline (∼100M parameters, identical data and training setup) reaches **27.08**—PAM is within ∼10%, while being fundamentally different in kind. We do not claim parity with transformers. What we do claim is that PAM represents a genuinely different approach—complex-valued, attention-free, phase-geometric retrieval with fixed recurrent state and $O(1)$-per-token inference—that has reached a level where continued research is warranted.

The transition from vector-state SSMs (interference-limited) to matrix-state PAM, and then to **interleaved** blocks with **RoPE** and an updated training recipe, validates that complex matrix states with phase-interference retrieval can serve as a viable sequence mechanism. The ∼10% gap to a matched transformer baseline, achieved without custom kernels and with 4× arithmetic overhead from complex computation, suggests that closing this gap through implementation optimization and architectural refinement is a realistic and worthwhile research direction.

A Hyperparameter Details

A.1 medium-pam-v3 configuration (reported)

```
preset:                 medium-pam-v3
```

```
vocab_size:           50,257
complex_dim:          384
num_layers:           16
single_bank:          True
bank_expand:          3 (CGU hidden = 1,152)
interleave_pam:       True
state_dim:            0 (SSM disabled)
pam_num_heads:        6
pam_head_dim:         64
pam_rope:             True
pam_qk_norm:          False
pam_fused_qkv:        True
gated_state_protection: True
dropout:              0.1
init_strategy:        orthogonal
```

A.2 medium-pam configuration (sequential baseline)

Same embedding and PAM head dimensions as above, but **no** interleaving, **no** RoPE, **no** fused QKV flag in the preset; training used **learning rate** 3×10^{-5} and **500** warmup steps (see Section 5.1). Full preset: `medium-pam` in `v6/config.py`.

A.3 Matched transformer baseline configuration

```
d_model:              672
n_layers:             12
n_heads:              12
d_ff:                 2688
total_parameters:     ~100.3M
dataset:              WikiText-103
tokenizer:            GPT-2 BPE (50,257 tokens)
seq_len:              2,048
batch_size:           3
optimizer:            AdamW
learning_rate:        1e-4
weight_decay:         0.01
warmup_steps:         1000
lr_schedule:          warmup + cosine decay
gradient_clip:        1.0
amp_dtype:            bf16 (automatic)
compile:              torch.compile (default mode)
hardware:             1x NVIDIA RTX 4090 (24GB)
```

A.4 Training configuration (medium-pam-v3)

```
dataset:              WikiText-103
tokenizer:            GPT-2 BPE (50,257 tokens)
seq_len:              2,048
batch_size:           3
```

```
epochs:              10
optimizer:           AdamW
learning_rate:       1e-4
weight_decay:        0.01
warmup_steps:        1000
lr_schedule:         warmup + cosine decay
gradient_clip:       1.0
amp_dtype:           bf16 (automatic)
compile:             torch.compile (default mode)
hardware:            1x NVIDIA RTX 4090 (24GB)
```

A.5 Parameter breakdown

Component	Parameters	Percentage
Embedding (tied with output)	∼38.6M	38.4%
CGU + PAM blocks (16 blocks)	∼61.7M	61.3%
LM head projection + norm	∼0.3M	0.3%
Total	**∼100.4M**	**100%**

Note: the embedding accounts for a large fraction because dim=384 with vocab_size=50,257 requires $50{,}257 \times 384 \times 2 = 38.6$M parameters (real + imaginary embedding tables). These are tied with the output layer.

B Reproducibility

Full source code and training scripts: `https://github.com/gowrav-vishwakarma/qllm2`.

Python 3.9+ (tested with 3.9.24); PyTorch 2.x with CUDA support; package management via `uv`.

The reported **29.95** validation PPL run is archived under `logs/v6/medium_pam_v3_rope_wikitext103_20260319` (`RUN_INFO.txt` lists git commit `31397f0` and full CLI). The matched transformer baseline (**27.08**) is archived under `logs/v6/transformer_baseline_wikitext103_20260323_140351_e87b8e4/` (git commit `e87b8e4`).

```
uv sync && uv sync --extra cuda
./scripts/run_v6_medium_pam_v3.sh
./scripts/run_v6_medium_pam.sh
./scripts/run_v6_medium_pam.sh --resume
```

Generation:

```
python -m v6.generate \
  --checkpoint checkpoints_v6_medium_pam_v3/best_model.pt \
  --prompt "In 1923 , the University of"
```

References

[1] Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *International Conference on Machine Learning*, 2016.

[2] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[3] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[4] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[5] Akira Hirose. *Complex-Valued Neural Networks*. Springer, 2012.

[6] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.

[7] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[8] Tony A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.

[9] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[10] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[11] Chiheb Trabelsi, Olexa Bilaniuk, Ying Zhang, Dmitriy Serdyuk, Sandeep Subramanian, João Felipe Santos, Soroush Mehri, Negar Rostamzadeh, Yoshua Bengio, and Christopher J. Pal. Deep complex networks. In *International Conference on Learning Representations*, 2018.

[12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.